```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from skimage.feature import hog
from skimage import exposure

from google.colab import files
```

```
uploaded = files.upload()
```

Choose Files bottle.avif

bottle.avif(image/avif) - 27428 bytes, last modified: 9/10/2026 - 100% done

```
Saving bottle.avif to bottle.avif
```

```
filename = list(uploaded.keys())[0]

print("Uploaded file:", filename)
```

```
Uploaded file: bottle.avif
```

```
image = cv2.imread(filename)

print("Image shape:", image.shape)
```

```
Image shape: (2000, 2000, 3)
```

```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
plt.figure(figsize=(8,6))

plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')

plt.show()
```




```
sift = cv2.SIFT_create()
```

```
keypoints, descriptors = sift.detectAndCompute(gray, None)

print("Number of keypoints detected:", len(keypoints))
print("Descriptor shape:", descriptors.shape)
```

```
Number of keypoints detected: 1053
Descriptor shape: (1053, 128)
```

```
sift_image = cv2.drawKeypoints(
    image_rgb,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)
```

```
plt.figure(figsize=(10,8))

plt.imshow(sift_image)
plt.title("SIFT Keypoints")
plt.axis('off')

plt.show()
```




```
plt.figure(figsize=(10,8))

plt.imshow(sift_image)
plt.title("SIFT Keypoints")
plt.axis('off')

plt.show()
```




```
hog_image = cv2.resize(gray, (128, 128))
```

```
plt.figure(figsize=(6,6))

plt.imshow(hog_image, cmap='gray')
plt.title("Grayscale Image for HOG")
plt.axis('off')

plt.show()
```



```
features, hog_visualization = hog(
    hog_image,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    visualize=True,
    block_norm='L2-Hys'
)
```

```
print("HOG feature vector length:", len(features))
```

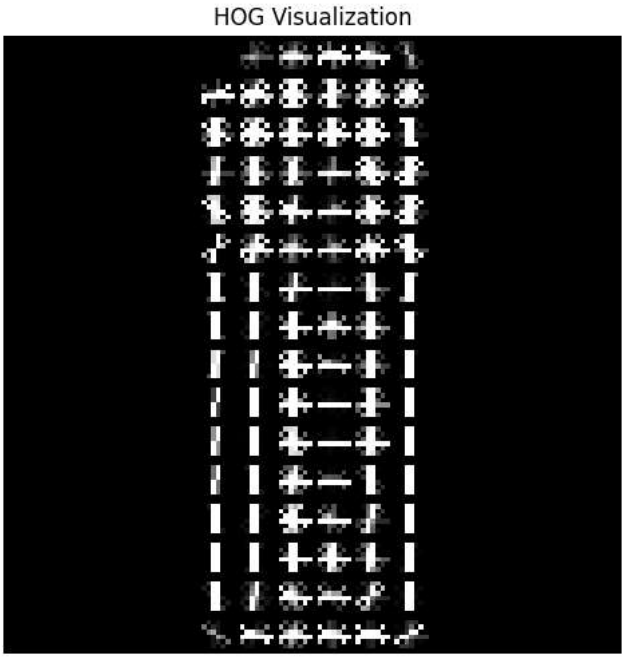
```
HOG feature vector length: 8100
```

```
hog_visualization = exposure.rescale_intensity(
    hog_visualization,
    in_range=(0, 10)
)
```

```
plt.figure(figsize=(8,6))

plt.imshow(hog_visualization, cmap='gray')
plt.title("HOG Visualization")
plt.axis('off')

plt.show()
```




```
comparison = {
    "Feature Type": ["SIFT", "HOG"],
    "Main Representation": [
        "Local keypoints and descriptors",
        "Gradient orientation distribution"
    ],
    "Scale Invariant": ["Yes", "No"],
    "Rotation Invariant": ["Yes", "Limited"],
    "Feature Type": ["Local", "Global/region-based"],
    "Typical Use": [
        "Image matching and object recognition",
```

```
            "Object detection and shape analysis"
        ]
    }

    for key, value in comparison.items():
        print(key, ":", value)
```

```
Feature Type : ['Local', 'Global/region-based']
Main Representation : ['Local keypoints and descriptors', 'Gradient orientation distribution']
Scale Invariant : ['Yes', 'No']
Rotation Invariant : ['Yes', 'Limited']
Typical Use : ['Image matching and object recognition', 'Object detection and shape analysis']
```

```
uploaded = files.upload()
```

Choose Files building1.jpg
building1.jpg(image/jpeg) - 19491 bytes, last modified: 9/10/2026 - 100% done
Saving building1.jpg to building1.jpg

```
uploaded2 = files.upload()
```

Choose Files building2.jpg
building2.jpg(image/jpeg) - 64193 bytes, last modified: 9/10/2026 - 100% done
Saving building2.jpg to building2.jpg

```
files_list = list(uploaded.keys()) + list(uploaded2.keys())

print(files_list)
```

```
['building1.jpg', 'building2.jpg']
```

```
img1 = cv2.imread(files_list[0])
img2 = cv2.imread(files_list[1])

gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
```

```
sift = cv2.SIFT_create()

kp1, des1 = sift.detectAndCompute(gray1, None)
kp2, des2 = sift.detectAndCompute(gray2, None)

print("Image 1 keypoints:", len(kp1))
print("Image 2 keypoints:", len(kp2))
```

```
Image 1 keypoints: 643
Image 2 keypoints: 1465
```

```
index_params = dict(
    algorithm=1,
    trees=5
)

search_params = dict(
    checks=50
)

flann = cv2.FlannBasedMatcher(
    index_params,
    search_params
)
```

```
matches = flann.knnMatch(
    des1,
    des2,
    k=2
)
```

```
good_matches = []

for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good_matches.append(m)

print("Good matches:", len(good_matches))
```

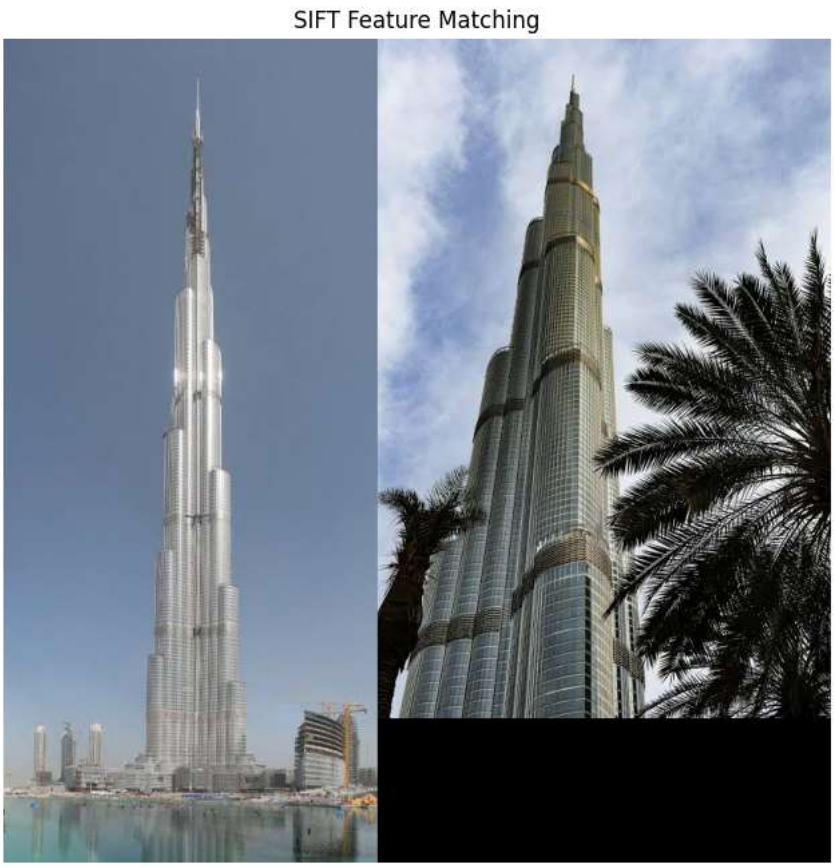
```
Good matches: 0
```

```
match_image = cv2.drawMatches(
    img1,
    kp1,
    img2,
    kp2,
    good_matches,
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)
```

```
plt.figure(figsize=(16,8))

plt.imshow(cv2.cvtColor(match_image, cv2.COLOR_BGR2RGB))
plt.title("SIFT Feature Matching")
plt.axis('off')

plt.show()
```




```
threshold = 20

if len(good_matches) >= threshold:
    print("The images show strong feature similarity.")
else:
    print("The images show weak feature similarity.")
```

```
The images show weak feature similarity.
```

```
plt.figure(figsize=(16,10))

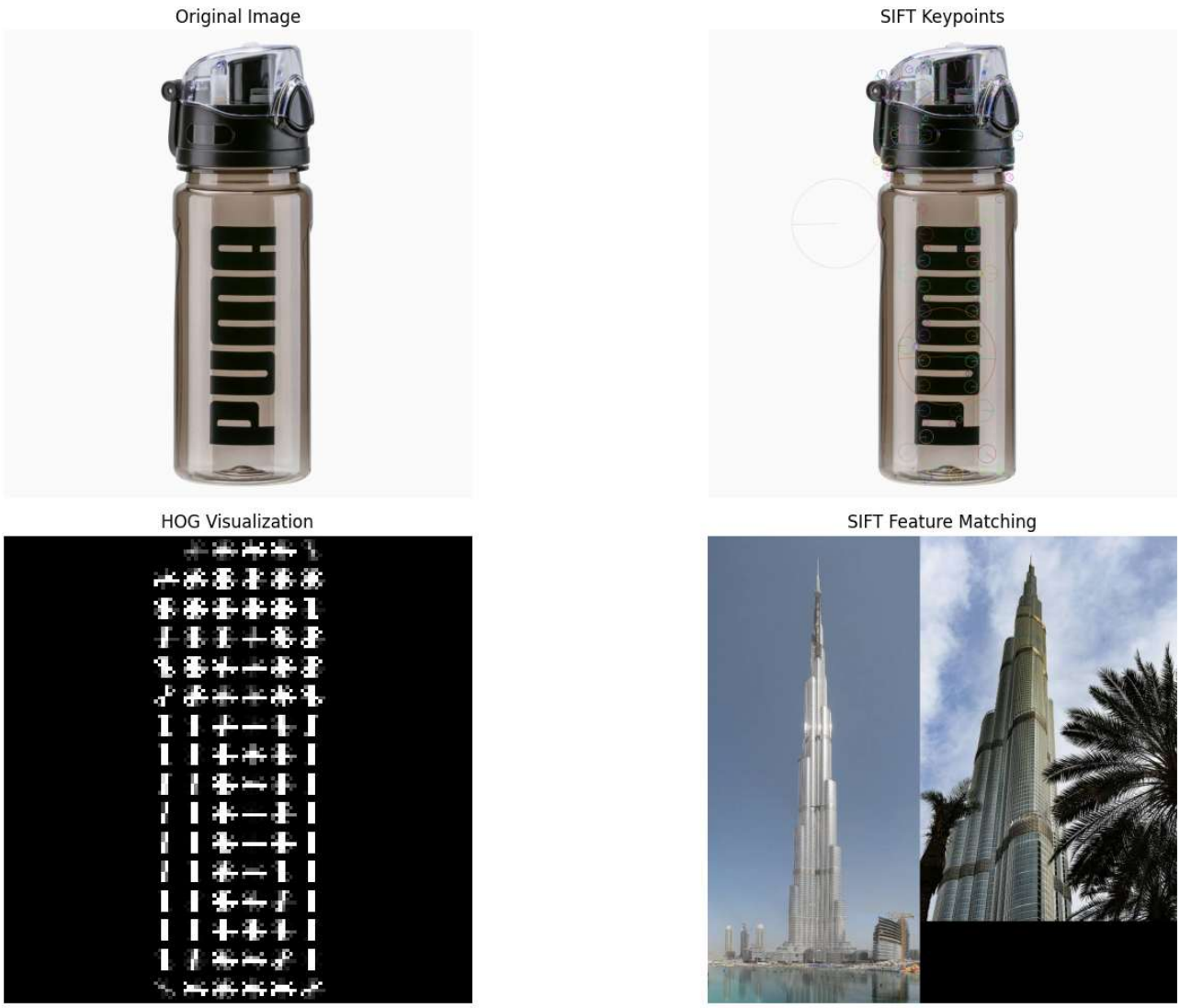
plt.subplot(2,2,1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')

plt.subplot(2,2,2)
plt.imshow(sift_image)
plt.title("SIFT Keypoints")
plt.axis('off')

plt.subplot(2,2,3)
plt.imshow(hog_visualization, cmap='gray')
plt.title("HOG Visualization")
plt.axis('off')

plt.subplot(2,2,4)
plt.imshow(cv2.cvtColor(match_image, cv2.COLOR_BGR2RGB))
plt.title("SIFT Feature Matching")
plt.axis('off')
```

```
plt.tight_layout()
plt.show()
```



PART H — THEORY QUESTION

Q1. What is feature extraction, and why is it important in computer vision?

Answer:

Feature extraction is the process of converting raw image data into meaningful characteristics or numerical descriptors that can be used by computer vision algorithms.

It is important because raw images contain large amounts of information. Extracting useful features makes tasks such as:

Object recognition Image matching Tracking Classification

more manageable.

Q2. Explain the working principle of SIFT.

Answer:

SIFT, or Scale-Invariant Feature Transform, detects distinctive image features and creates descriptors for those features.

The basic process involves:

Detecting distinctive keypoints. Identifying their scale. Assigning orientations. Generating descriptors that represent the local image region around each keypoint.

The resulting descriptors can be compared between images to find corresponding features.

SIFT is designed to be robust to changes in scale and rotation.